

Estruturas de Repetição

O objetivo principal de um loop é automatizar uma tarefa repetitiva. Em vez de escrever o mesmo código várias vezes, o usuário escreve uma vez dentro de um laço(loop) e define a regra para sua repetição.

FOR : Repetição sobre uma Coleção Finita

Significado: Use o for quando você tem um conjunto definido de itens e quer executar uma ação para cada um deles. Pense nele como uma linha de montagem: cada item da esteira (a sequência) passa por uma estação (o bloco de código) para ser processado.

Exemplo Prático:

Calcular o total de um carrinho de compras

Imagine que você tem uma lista de produtos, cada um representado por um dicionário com nome e preço. Você precisa calcular o valor total da compra.

```
carrinho_de_compras = [  
    {"nome": "Teclado", "preco": 150.00},  
    {"nome": "Mouse", "preco": 80.50},  
    {"nome": "Monitor", "preco": 950.00},  
    {"nome": "Cadeira Gamer", "preco": 1200.00}  
]  
  
total_compra = 0.0  
  
# Para cada 'produto' na lista 'carrinho_de_compras'...  
for produto in carrinho_de_compras:  
    # ...pegue o preço desse produto e some ao total.  
    preco_item = produto["preco"]  
    total_compra += preco_item  
    print(f"Adicionando {produto['nome']} (R$ {preco_item:.2f}) ao total.")  
  
print("=" * 30)  
# A formatação :.2f garante que o valor seja exibido com duas casas decimais.  
print(f"O valor total da compra é: R$ {total_compra:.2f}")
```

O laço **for** é perfeito porque temos uma coleção finita (a lista `carrinho_de_compras`) e queremos fazer exatamente a mesma coisa (somar o preço) para cada item

Exemplo Prático: Enviar e-mails personalizados

Você tem uma lista de e-mails de usuários e quer enviar uma saudação personalizada para cada um.

```
lista_de_emails = ["ana@email.com", "bruno.costa@email.com", "carla123@email.com"]

# Para cada 'email' na 'lista_de_emails'...
for email in lista_de_emails:
    # ...crie uma mensagem personalizada e simule o envio.
    nome_usuario = email.split('@')[0] # Pega a parte do e-mail antes do @
    mensagem = f"Olá {nome_usuario}, obrigado por se inscrever em nossa newsletter!"

    # Aqui iria o código real para enviar o e-mail
    print(f"Enviando para {email}: '{mensagem}'")

print("-" * 30)
print("Todos os e-mails foram enviados com sucesso!")
```

Neste exemplo o **for**, automatiza a tarefa de personalização e envio. Sem ele, você teria que copiar e colar o código para cada e-mail da lista.

Estruturas de Repetição

WHILE: Repetição Baseada em uma Condição

Significado: Use o **while** quando a repetição depende de uma condição que precisa ser verificada a cada passo.

O loop continua enquanto a condição for verdadeira.

É ideal para situações, onde você não sabe de antemão quantas repetições serão necessárias.

Exemplo Prático: Jogo de Adivinhação

Jogo simples onde o computador escolhe um número e o usuário tenta adivinhar. O jogo continua até que o usuário acerte.

```
import random

numero_secreto = random.randint(1, 20)
palpite = 0

print("Jogo de Adivinhação! Tente adivinhar o número entre 1 e 20.")

# Enquanto o 'palpite' do usuário for diferente do 'numero_secreto'...
while palpite != numero_secreto:
    # ...peça um novo palpite.
    try:
        palpite = int(input("Qual o seu palpite? "))

        if palpite < numero_secreto:
            print("Muito baixo! Tente novamente.")
        elif palpite > numero_secreto:
            print("Muito alto! Tente novamente.")
    except ValueError:
        print("Por favor, digite um número válido.")

print("=" * 30)
print(f"Parabéns! Você acertou! O número secreto era {numero_secreto}.")
```

O **while** é essencial porque não sabemos quantas tentativas o jogador levará. O loop roda um número indeterminado de vezes, parando apenas quando a condição `palpite != numero_secreto` se torna falsa.

Exemplo Prático: Consumir itens de uma fila de tarefas

Imagine um sistema que processa tarefas de uma fila. Ele deve continuar processando enquanto houver tarefas na fila.

```
fila_de_tarefas = ["Imprimir relatório A", "Enviar e-mail de marketing", "Fazer backup do banco de dados", "Atualizar sistema"]

print("Iniciando processamento da fila de tarefas...")

# Enquanto a 'fila_de_tarefas' não estiver vazia...
while len(fila_de_tarefas) > 0:
    # ...pegue a próxima tarefa da fila (a primeira).
    tarefa_atual = fila_de_tarefas.pop(0) # .pop(0) remove e retorna o primeiro item

    print(f"Processando: '{tarefa_atual}'...")
    # Simula o tempo de processamento
    # import time; time.sleep(1)

    print(f"Restam {len(fila_de_tarefas)} tarefas na fila.")

print("=" * 30)
print("Todas as tarefas foram processadas. Fila vazia!")
```

Estruturas de Repetição

BREAK: Parar a busca quando o item é encontrado

Significado: Interromper o laço imediatamente, mesmo que a condição do while ainda seja verdadeira ou que ainda haja itens no for.

Exemplo Prático: Verificar se um nome de usuário já existe em um banco de dados. Assim que você o encontra, não precisa continuar verificando o resto da lista.

```
usuarios_cadastrados = ["ana", "carlos", "beatriz", "daniel", "elaine"]
novo_usuario = "beatriz"
usuario_existe = False

print(f"Verificando se o usuário '{novo_usuario}' já existe...")

for usuario in usuarios_cadastrados:
    print(f"Comparando com '{usuario}'...")
    if usuario == novo_usuario:
        print("Usuário encontrado!")
        usuario_existe = True
        break # Sai do laço, pois não há necessidade de continuar a busca

if usuario_existe:
    print(f"O nome de usuário '{novo_usuario}' já está em uso.")
else:
    print(f"O nome de usuário '{novo_usuario}' está disponível.")
```

O break torna o código mais eficiente, pois ele para de gastar tempo de processamento assim que o objetivo é alcançado.

Estruturas de Repetição

continue: Ignorar dados inválidos em um processamento em lote

Significado: Pular o resto do código da iteração atual e ir diretamente para a próxima.

Exemplo Prático: Calcular a média de notas, mas ignorando valores inválidos (como notas negativas).

```
notas_alunos = [8.5, 9.0, -1.0, 7.2, 10.0, -5.0, 6.8, 11]
soma_notas_validas = 0.0
quantidade_notas_validas = 0

print("--- Calculando média de notas válidas---")

for nota in notas_alunos:
    if nota < 0 or nota > 10:
        print(f"Nota inválida encontrada ({nota}). Ignorando...")
        continue # Pula para a próxima nota, ignorando as linhas abaixo

    # Este código só executa para notas válidas
    soma_notas_validas += nota
    quantidade_notas_validas += 1

media = soma_notas_validas / quantidade_notas_validas
print("=" * 30)
print(f"Média das notas válidas: {media:.2f}")
```

O continue permite que o programa lide com dados "sujos" de forma elegante, sem interromper o processamento dos dados válidos.